Memorandum

The task is to create a partially autonomous system for session catering.

Key Requirements

- Create a structured and organised database to replace the csv/pdf mess
- Account for student dietary requirements and special dietary requirements (if any)
- Implement functionality to get required dietary needs for a specific session
- Implement functionality to generate an order for a specific session

Extra Requirements

- Account for manager feedback in meal choices
- Produce a simple and readable output

Philosophy

During this competition, I stuck by a few basic principles.

- Build in stages, get something working then extend
- Only implement LLM usage when absolutely necessary, every LLM call introduces uncertainty
- Focus on the logic behind the scenes and stick to basic presentation

First stage – database

Given I plan to produce a code solution, I chose Supabase as my database, it is simple, robust and supports SQL, which I plan on using. During this stage I spent an hour or two learning about Supabase including the client libraries, various methods of connecting to databases and higher-level database concepts such as row-level security. I also learned more about the specific flavour of SQL that Supabase runs (PostgreSQL).

After familiarising with Supabase, I spent a lot of time iteratively extending and critiquing my database schema. Starting with just students, I gradually expanded outwards, while considering requirements in the resources and thinking about what assumptions I could reasonably make until it was at a stage where functionality could be implemented.

Since the database forms the concrete truth layer behind the entire system, I spent a lot of time ensuring I was happy with my database schema. Any problems here would cause many issues upstream.

Second stage – functionality and database extensions

After reaching a minimum viable database, I decided to start iteratively implementing functionality in accordance with my philosophy of building in stages. As such, I was able to quickly and successfully implement a simple system that was able to generate a catering order for a given group of students with dietary requirements that fit the tags provided by caterers (NF, GF, etc).

From here, I listed out all the edge cases in priority order and iteratively extended my solution to handle them. I chose this approach since it allowed me to quickly improve my solution and it ensure that if I need to stop early, I have already done the most important parts.

Third stage – output formatting

Once I reached a stage where I could produce an order list for a specific session. I needed a way to display it. Given the tight deadline, I decided to keep the presentation simple and produce a pdf containing all the relevant information. This allowed me to iterate quickly since the output and formatting logic was simple.

Fourth stage – LLM integration

In keeping with my philosophy, I left LLM integration to the last stage, only after I was confident that I had completed all important parts of the system that could be implemented algorithmically, I filled in the required gaps with an LLM. As such, LLM calls were only made where natural language was provided and some sort of result needed to be extracted – namely, interpreting manager feedback and special student dietary requirements that didn't fit into a standard dietary tag.

This allowed me to contain the LLM calls to minimal, well-scoped tasks, which helps reduce the margin for error and allows me to implement guardrails and response validation. This allowed me to use a smaller, lighter and free LLM model, making the system faster and cheaper. This also gave me the opportunity to learn how to download, host and connect to a locally deployed LLM.

Fifth stage – Validation and clean up

Once I had implemented all the functionality, I created test cases and cleaned up the code I already had, in which I managed to catch a few bugs and tidy up my code into a relatively clean and extensible solution.